

Capítulo 9

Combate por Turnos, UI y Guardado de Partidas

“Lo que hacía que un Ultima fuera un Ultima era la elaboración minuciosa de la historia, el cuidado al crear esos trasfondos, y el cuidado de crear eventos socialmente relevantes para ti.”

—Richard Garriott, *The Ultima Codex*, 2013

Princess (Capítulo 8) introdujo la pila de estados como una pieza de infraestructura que su propia mecánica todavía no necesitaba usar a fondo. Ultimate Fantasy es el juego que finalmente la exige en código real. De paso se convierte, en escala, en el más grande del libro hasta este punto: un mundo navegable dividido en regiones, un sistema de entidades de juego (véase entidad de juego) compartido entre el jugador, sus compañeros de equipo, los aldeanos con los que se puede hablar y los enemigos, un combate por turnos completo con selección de acción y de objetivo, y una interfaz de menús anidados para combate, inventario de acciones y progreso de nivel. Concretamente, es el primer juego del libro construido sobre `StateStack` (Sección 8.2 del Capítulo 8) en lugar de una única máquina de estados, ya que casi cada pantalla del juego se apila sobre la anterior en vez de reemplazarla. El objetivo del capítulo es estudiar cómo se organiza el código cuando un juego deja de caber cómodamente en un puñado de archivos. También busca mostrar cómo `gale.ui` resuelve el problema, transversal a cualquier juego con menús, de construir una interfaz sin repetir la lógica de “¿a quién le corresponde este click o esta tecla?” en cada pantalla.

9.1. Contexto histórico: de un mundo persistente en el PC a un combate por turnos japonés

Ultimate Fantasy combina, con nombres propios, dos linajes distintos del CRPG¹: la tradición estadounidense de mundo abierto explorable inaugurada por la saga *Ultima*, y la tradición japonesa de combate por turnos y elenco narrativo inaugurada por *Final Fantasy*.

¹juego de rol de computadora, del inglés *Computer Role-Playing Game*

Ninguno de los dos juegos originales tiene afiliación alguna con este libro ni con *Ultimate Fantasy*, pues se mencionan aquí únicamente para situar de dónde viene cada una de las dos tradiciones que este capítulo combina.

Ultima III: Exodus (Origin Systems, 1983) fue el primer título publicado por la compañía fundada ese mismo año por Richard Garriott junto a su hermano Robert y su padre Owen. Fue también el primero de la saga en incorporar un grupo completo de hasta cuatro aventureros controlados simultáneamente, además de pantallas de combate independientes del mapa de exploración. El historiador Matt Barton lo señala como un hito fundacional del CRPG por consolidar el mapa por losetas, el sistema de combate por turnos de grupo, y una narrativa que trascendía la simple mazmorra para articular un mundo persistente con trama propia (Barton, 2008; Loguidice & Barton, 2009).

La influencia de estos CRPG occidentales en Japón está documentada por sus propios protagonistas. Hironobu Sakaguchi, diseñador en Square, ha declarado en entrevistas que *Ultima* y *Wizardry* estaban entre sus juegos favoritos y una fuente explícita de inspiración. De *Wizardry* retomó la idea del grupo de personajes controlado por el jugador, y de *Ultima*, la ambición de un mundo abierto explorable (Deken, 2021). El resultado, *Final Fantasy* (Square, 1987), contribuyó a popularizar en Japón, y después en Occidente, el subgénero conocido como JRPG². Conviene precisar un dato que durante años circuló como verdad asumida, con base en una entrevista reciente por el 35.º aniversario de la saga: la leyenda de que el juego se llamó así porque iba a ser el último de Sakaguchi, o el último de Square ante una quiebra inminente. El propio Sakaguchi desmintió esa versión, reconociendo incluso haber contribuido él mismo a perpetuarla en el pasado. Explicó que el nombre surgió de un proceso más prosaico: el equipo buscaba un título abreviable con dos letras, a semejanza de cómo *Dragon Quest* se abreviaba coloquialmente. La primera opción, “Fighting Fantasy”, se descartó por motivos de marca registrada, y el equipo optó, como “último recurso”, por “Final Fantasy” («Final Fantasy at 35: Hironobu Sakaguchi on the Origin of the Name, and How the Myth of Square’s “Final” Game Came to Be», 2022). El episodio es pedagógicamente útil por lo que ilustra sobre el oficio mismo de la historia de los videojuegos: incluso los mitos fundacionales sostenidos durante décadas por sus propios creadores requieren verificarse con fuentes primarias actualizadas antes de repetirse como dato cierto.

9.2. Conceptos nuevos

- **Mundo dividido en regiones:** cinco pantallas (centro/pueblo y norte/sur/este/occidente), cada una un `TileMap` llenado por código en vez de cargado desde `Tiled`, con una capa de colisión (la cerca perimetral) separada de la capa visual (césped, flores, pasto alto).
- **Jerarquía de entidades compartida:** qué tienen en común un jugador, un com-

²juego de rol de origen japonés, del inglés *Japanese Role-Playing Game*



Figura 9.1: Hironobu Sakaguchi, creador de *Final Fantasy*, en la Game Developers Conference de 2007.

Fotografía de “Gamerscore Blog” (Flickr), licencia CC BY-SA 2.0, vía Wikimedia Commons.



Figura 9.2: Richard Garriott, creador de la saga *Ultima*.

Fotografía de Rob Fahey, licencia CC BY-SA 2.0, vía Wikimedia Commons.

pañero de equipo, un aldeano (NPC) y un enemigo (posición, animación, máquina de estados) frente a qué es exclusivo del combate (salud, estadísticas, fórmulas de daño) y cómo modelar esa diferencia con dos clases base en vez de una sola.

- **Combate por turnos** como una máquina de estados propia, apilada sobre el estado de combate: menú de acción, selección de objetivo, resolución del daño, verificación de victoria/derrota, cesión del turno al siguiente combatiente.
- **Menús anidados** (menú de acción dentro de menú de combate dentro de la pantalla de batalla, que a su vez está apilada sobre el mundo) como una aplicación más profunda de la pila de estados de la Sección 8.2.
- **UI** como una capa de widgets sobre el *input handler*: cómo un widget decide si un evento le corresponde, cómo un contenedor compone y enruta eventos a sus hijos, y cómo un tema centraliza la apariencia.

- **Persistencia de partidas:** la posibilidad de guardar la partida que el Capítulo 8 dejó anunciada al hablar del Family Computer Disk System de 1986 se implementa aquí de verdad, en un menú de pausa que, como corresponde a este capítulo, es una pieza más de la pila de estados.

9.3. El mundo: regiones, capas de tiles y colisión

A diferencia de Super Martian (Capítulo 7), donde el mapa se diseña en Tiled y se carga con `load_tiled_map`, el mundo de Ultimate Fantasy no tiene archivos `.tmx`: cada `Region` se genera por código, en `src/world/Region.py`, escribiendo directamente sobre un `TileMap` – el mismo tipo que Super Martian llena a partir de un JSON, aquí llenado a mano. La región completa vive en una sola instancia con tres capas apiladas, en orden de renderizado: una base de césped, una cerca perimetral, y una decoración (flores, pasto alto o aldeanos, según si la región es el pueblo o no). La idea de “grilla de identificadores de tile (*gid*) más una lectura de colisión separada” es la misma que en Super Martian, y los `tile_id` de `settings.TILE_IDS` sirven directamente como gids, porque el `Tileset` de la hoja de tiles arranca, igual que en Super Martian, en `first_gid=1`. Lo único que cambia es que aquí no hace falta un editor visual, porque el diseño de cada región es un patrón geométrico simple (borde + puertas + relleno aleatorio) que es más corto de generar por código que de dibujar tile por tile:

```

1 self.tilemap = TileMap(
2     settings.TILE_SIZE, settings.TILE_SIZE, self.tile_width, self
3     .tile_height
4 )
5 self.tilemap.add_tileset(settings.TILESET)
6
7 base = self.tilemap.add_layer("base")
8 for y in range(1, height + 1):
9     for x in range(1, width + 1):
10         base[y - 1][x - 1] = random.choice(TILE_IDS["grass"])

```

`settings.TILESET` es un único `Tileset` compartido por toda región y por el escenario de combate (Sección siguiente): se construye una sola vez, al cargar `settings.py`, a partir de la misma hoja de sprites que antes recortaba `settings.frame("tiles", ...)`.

`Region._create_maps` construye las tres capas en pasos claramente separados: primero césped aleatorio en toda la grilla (como en el fragmento anterior), luego el borde de cerca (con esquinas y bordes rectos distintos), luego un “tallado” de hasta cuatro puertas de dos tiles de ancho (una por cada lado que la región declare como `_gate: True` en su definición), y por último la capa decorativa. La colisión nunca se calcula en abstracto, sino que se pregunta directamente sobre la capa de cerca, en el momento de intentar mover al grupo, exactamente como hacía Super Martian con su capa de colisión:

```

1 region = party.world.current_region()

```

```

2  if region.tilemap.get_gid("fence", to_y - 1, to_x - 1) !=
    settings.TILE_IDS["empty"]:
3      party.change_state("idle")
4      return

```

Salir de los límites de la grilla no es un choque contra un muro sino una transición de región (`World.move`), que dispara un `FadeOutState/FadeInState` (estados de la propia pila, iguales en espíritu a los del Capítulo 8) antes de reposicionar al grupo en el borde opuesto de la región vecina. Pisar una casilla de pasto alto en movimiento, en cambio, tiene una probabilidad de 1 en 10 de disparar un encuentro aleatorio que empuja un `BattleState` completo sobre la pila, sin abandonar el `PlayState` subyacente:

```

1  gid = region.tilemap.get_gid("grass", to_y - 1, to_x - 1)
2  if gid != settings.TILE_IDS["tall-grass"]:
3      return False
4  if random.randint(1, 10) != 1:
5      return False
6  self._trigger_encounter()

```

El grupo (`Party`) se mueve en formación de “serpiente”, pues solo el líder (el primer personaje vivo) decide la dirección. Cada seguidor gira para mirar hacia quien tiene delante y, al terminar el paso, se desliza a la posición que esa persona ocupaba antes de moverse. Esto reutiliza la idea de `tween` del Capítulo 6 (interpolando `x/y` con `Timer.tween` en vez de saltar de `tile` en `tile`) para que el desplazamiento se vea continuo aunque la lógica de colisión razone en coordenadas de grilla discretas.

9.4. Una jerarquía de entidades en dos niveles

El juego necesita modelar cuatro tipos de “cosas que existen en el mundo o en combate”: el jugador controla hasta cuatro `Character`; los aldeanos son `NPC`; los monstruos son `Enemy`; y el propio `Party` agrupa a los `Character` vivos. No todas comparten lo mismo: un `NPC` tiene posición y animación pero jamás entra en combate, mientras que un `Enemy` sí necesita salud, ataque, defensa y magia pero nunca camina por el mundo. `src/entity/` resuelve esto con dos niveles de herencia en vez de una sola clase con banderas:

- `Entity` (`src/entity/Entity.py`) es la base de *cualquier* cosa dibujable con posición y animación: nombre, dirección, textura, diccionario de animaciones, posición en tiles (`map_x/map_y`) y en píxeles (`x/y`, derivada de la anterior), y una `state_machine` que `Entity` declara pero nunca construye. Cada subclase, o quien la instancia, decide qué estados tiene.
- `BattleEntity` (`src/entity/BattleEntity.py`) extiende `Entity` añadiendo exactamente lo que hace falta para participar en combate: `hp/attack/defense/magic` (y sus versiones `base_*`), una lista de acciones disponibles, y los métodos `damage/heal/compute_attack/`

`compute_defense/compute_healing` que implementan las fórmulas de daño con un componente aleatorio.

- **Character** extiende **BattleEntity** agregando lo que solo el jugador tiene: variables individuales de crecimiento (`hpiv`, `attackiv`, `defenseiv`, `magiciv`, tiradas en 1d6 contra cada una en cada subida de nivel) y una curva de experiencia (`exp_to_level = nivel2 × 10 × 1.1`).
- **Enemy** extiende **BattleEntity** sin añadir nada, pues un enemigo son exactamente los atributos de combate, sin progresión ni inventario.
- **NPC** extiende **Entity** directamente (nunca **BattleEntity**) y solo agrega `on_interact`, que sorteas una línea de diálogo al azar.

La Figura 9.3 dibuja esta jerarquía como un diagrama de clases antes de leerla en código: dos niveles de herencia, con **Entity** como raíz común y **BattleEntity** como la única clase que agrega los atributos de combate, de la que cuelgan **Character** y **Enemy**.

Esta separación en dos niveles es la respuesta concreta al ejercicio de diseño más importante del capítulo. En vez de una única clase **Entity** con campos opcionales para salud/ataque que los NPC dejarían en `None`, el árbol de herencia hace que un NPC *no pueda* tener `hp` por accidente. Además, cualquier función que opere sobre combate (el bucle de turnos, el cálculo de daño) puede declarar su parámetro como **BattleEntity** y confiar en que `compute_attack` existe, sin necesidad de comprobarlo:

```

1 class BattleEntity(Entity):
2     def __init__(self, definition):
3         super().__init__(definition)
4         self.hp = self.base_hp
5         self.attack = self.base_attack
6         ...
7
8     def damage(self, amount: float) -> None:
9         self.current_hp -= amount
10        if self.current_hp <= 0:
11            self.dead = True
12
13    def compute_attack(self) -> int:
14        return math.floor(
15            random.random() / 2 * self.attack
16            + random.random() / 4 * self.magic
17        )

```

Nótese que ni **Character** ni **Enemy** sobrescriben `compute_attack`, `damage` o `heal`, ya que todo el código de combate (`TakeTurnState`, `SelectActionState`) llama a estos métodos sobre *lo que sea* que reciba como `entity/target` sin preguntar de qué clase es. Esa es, en la práctica, la respuesta al segundo ejercicio propuesto de la versión anterior de este

capítulo: la interfaz mínima para participar en combate es “tener `dead`, `compute_attack`, `compute_defense`, `damage` y `heal`”. Esa interfaz la cumplen `Character` y `Enemy` por herencia común de `BattleEntity`, sin que ninguna de las dos la reimplemente.

El movimiento por el mundo, en cambio, se modela con máquinas de estados por entidad (`src/states/entity/`), donde `EntityBaseState` es la base común y renderiza el sprite vía `entity.render_sprite`. De ella cuelgan `CharacterIdleState/CharacterWalkState` (cambian de animación según dirección), `NPCIdleState` (animación fija más una oscilación de mirada aleatoria) y `EnemyBattleState` (solo mantiene la animación “default” mientras `TakeTurnState` decide qué ataque ejecuta). El propio `Party` tiene su *propia* máquina de estados (`PartyIdleState/PartyWalkState`, en `PartyBaseState`) separada de la de cada `Character`. La del grupo decide *si* hay que moverse (leyendo qué teclas de dirección están sostenidas) y resuelve colisión/encuentros/transición de región; la de cada personaje solo decide qué animación mostrar. Es la misma separación de responsabilidades por capas que ya apareció en capítulos anteriores, aplicada aquí dos veces: una vez por entidad y otra vez para el grupo que las coordina.

9.5. El combate como máquina de estados apilada

Cuando el pasto alto dispara un encuentro, `PlayState` empuja un `BattleState` sobre la pila (Sección 8.2). El propio `BattleState` tiene su *propia* `StateMachine` interna (no otra `StateStack`). Es decir, `self.state_machine` en `BattleState` es, de hecho, el mismo objeto que la `StateStack` recibida como argumento del constructor de todos los estados que se apilan *sobre* el combate (`BattleMenuState`, `SelectActionState`, `SelectTargetState`, `BattleMessageState`, `TakeTurnState`). Por eso, estos navegan con `push/pop` exactamente como cualquier otro estado del juego. Antes de ver esa cadena de estados reales, conviene fijar la idea en un pseudocódigo de alto nivel, donde el bucle de turno, despojado de nombres de clase y de detalles de la pila, no es más que “recorrer a cada combatiente vivo, en orden, y resolver su turno”:

```
# Pseudocódigo (sintaxis informal, no es código Python real)
para cada bando (jugadores, luego enemigos):
    para cada combatiente vivo del bando, en orden:
        si el combatiente es controlado por el jugador:
            esperar seleccion de accion (menu de acciones)
            si la accion requiere objetivo:
                esperar seleccion de objetivo
        si no (es un enemigo):
            la IA elige una accion al azar
            la IA elige un objetivo vivo al azar
    resolver la accion (danio o curacion, con animacion)
    verificar victoria: ¿todos los enemigos muertos?
```

```

    verificar derrota: ¿todo el grupo muerto?
    si victoria o derrota: terminar el combate
    siguiente combatiente

```

La Figura 9.4 traduce ese pseudocódigo a la mecánica real de pila antes de leerla en prosa: cada paso no es una espera bloqueante sino un **push** concreto sobre la misma **StateStack**. La barra de activación de cada estado permanece abierta, sin haber hecho todavía **pop**, mientras el siguiente estado se apila encima, dejando ver cómo la profundidad de la pila crece con cada pantalla nueva en vez de reemplazar a la anterior.

Esa es, en esencia, la responsabilidad completa de **TakeTurnState**. En el código real, sin embargo, cada paso (“esperar selección de acción”, “esperar selección de objetivo”) no es una espera bloqueante sino un nuevo estado (**SelectActionState**, **SelectTargetState**) empujado sobre la pila, que le cede el control de vuelta a **TakeTurnState** mediante un *callback* al completarse. La secuencia completa de un turno, ya con los nombres reales de esos estados, se arma así:

1. **BattleState.enter** construye el escenario de combate (otro **TileMap**, con capas “base”/“grass” generadas igual que una región pero con tamaño fijo **BATTLE_WIDTH** × **BATTLE_HEIGHT**, y desplazado en pantalla dibujándolo sobre una subsuperficie recortada en vez de con un offset propio como antes) y coloca al grupo y a 3–5 enemigos aleatorios de la región (o, con 10 % de probabilidad en la región oeste, al jefe final más dos enemigos comunes) en posiciones prefijadas. Además crea una **ProgressBar** de vida (y, para el grupo, de experiencia) por cada combatiente vivo.
2. Tras el mensaje inicial (“¡Ha aparecido una horda de criaturas!” seguido de “¡Adelante, {nombres}!”), se empuja **BattleMenuState**: un menú de dos opciones, *Fight* y *Run*.
3. *Fight* reemplaza ese menú por **TakeTurnState**, que recorre a cada personaje vivo (en orden de posición) empujando, para cada uno, un **SelectActionState** —el menú de acciones propias de esa entidad, leído de **entity.actions**, con un ítem final fijo “Nothing” para pasar el turno—. Seleccionar una acción que requiere objetivo (**require_target**) empuja a su vez **SelectTargetState**, que mueve un cursor entre los objetivos vivos con izquierda/derecha y confirma con **enter**. Una acción de área (como un hechizo curativo grupal), en cambio, resuelve directamente sobre todos los objetivos vivos sin pedir selección.
4. Resuelta la acción (**action["func"]**, tomada de las definiciones en **src/definitions/entity.py**: por ejemplo **_character_attack** o **_character_heal_aoe**), se anima la barra de vida del objetivo con **Timer.tween** (Capítulo 6) y se empuja un **BattleMessageState** con el resultado, que al cerrarse cede el turno al siguiente personaje.
5. Terminados los personajes, **TakeTurnState** recorre a los enemigos vivos, donde cada uno elige una acción propia *al azar* (la IA completa del juego) garantizando un

objetivo vivo. Si es un jefe, además tiene 1 en 3 de probabilidad de encadenar un segundo ataque en la misma ronda (hasta tres seguidos).

6. Tras cada ronda completa se verifica la condición de fin: si todos los enemigos están muertos se dispara la secuencia de victoria (experiencia, posible subida de nivel con `StatsMenuState` mostrando el incremento de cada estadística, y un `fade` de regreso al mundo); si todo el grupo está muerto se dispara `GameOverState`. Ninguna de las dos condiciones se comprueba “en general”, sino que se pregunta explícitamente `all(enemy.dead for enemy in ...)` y `all(character.dead for character in ...)` en los puntos exactos donde el resultado de una acción pudo haber matado al último combatiente del bando contrario.

Esta cadena —menú de acción → selección de objetivo → resolución → verificación → siguiente turno— es, en sí misma, la “máquina de estados de combate” del capítulo: cada paso es un estado apilado que se `pop`-ea al terminar su trabajo y, típicamente, empuja el siguiente antes de desaparecer. *Ninguno* de estos estados conoce el flujo completo: `SelectActionState` no sabe qué pasó antes ni qué pasará después de que su `callback on_action_selected` se dispare. Solo `TakeTurnState` conoce el orden global, pasado como parámetros (`index`, `on_close`) de una función a la siguiente. Añadir un nuevo tipo de acción, por ejemplo, un objeto curativo usable desde un inventario dentro del propio combate, no requiere tocar `TakeTurnState` en absoluto. Basta con que ese objeto aparezca como una entrada más en la lista que arma `SelectActionState`, con su propio `action["func"]`.

9.6. Menús anidados: una pila dentro de la pila

El árbol de estados que un combate puede llegar a acumular sobre la pila, en su punto más profundo, es: `PlayState` (mundo, en pausa) → `BattleState` → `TakeTurnState` → `SelectActionState` → `SelectTargetState`. Cinco niveles, cada uno resuelto por completo con las mismas dos operaciones (`push/pop`) de la Sección 8.2, y cada uno dibuja solo lo que le corresponde: `BattleState.render` pinta el fondo y los combatientes; `SelectActionState.render` agrega el panel de menú *encima*; `SelectTargetState.render` agrega el cursor de objetivo *encima de eso*. Como la `StateStack` renderiza toda la pila de abajo hacia arriba pero solo entrega `on_input` al estado del tope, el jugador ve el campo de batalla completo en todo momento aunque solo pueda interactuar con el menú activo. `StatsMenuState` (mostrado tras una subida de nivel) y el propio `BattleMenuState` inicial siguen exactamente el mismo patrón. Son, en esencia, la misma idea de “inventario dentro de menú de combate dentro de pantalla de mundo” mencionada en los conceptos nuevos del capítulo, sea el contenido del menú una lista de acciones, de objetivos o de estadísticas ganadas.

9.7. Cómo lo implementa gale: gale.ui

Toda la interfaz descrita arriba, los menús de combate, las barras de vida y experiencia, el panel de fondo del menú, se construye con `gale.ui`, un módulo que este libro no había presentado todavía. Su capa inferior es la misma de siempre, es decir, `gale.input_handler` (Sección 5.2 del Capítulo 5), que entrega a cada estado los mismos eventos de teclado/mouse normalizados que Breakout ya usaba directamente. `gale.ui` agrega, por encima de esa capa, una jerarquía de *widgets* que saben dibujarse y decidir por sí mismos si un evento les corresponde. Así, un juego con muchas pantallas de menú no tiene que reimplementar esa lógica de enrutamiento en cada una.

La Figura 9.5 adelanta, en forma de diagrama, la idea central de todo `gale.ui` antes de leerla en código: un evento no se transmite a *todos* los widgets, sino que un contenedor lo enruta, dirigiéndolo únicamente al hijo al que efectivamente le corresponde.

9.7.1. Widget: el contrato común

Toda la jerarquía —`Panel`, `Button`, `ListView`, `Container`, ...— hereda de una única clase base, `Widget`. Su constructor fija posición, tamaño, un tema opcional y dos banderas, `visible` y `enabled`, que se leen en dos momentos distintos. `visible` decide si el widget se dibuja y es alcanzable por el enrutamiento de eventos. `enabled` decide, suponiendo que sí es alcanzable, si además reacciona a ellos: un widget deshabilitado igual se dibuja, típicamente con `theme.disabled_color`, pero ignora clicks y teclas. El contrato que cualquier subclase puede sobrescribir son cinco métodos:

```

1  def on_mouse_motion(self, position):
2      self.hovered = self.contains(position)
3
4  def on_mouse_click(self, position, data) -> bool:
5      return False
6
7  def on_confirm(self) -> bool:
8      return False
9
10 def on_navigate(self, direction) -> bool:
11     return False

```

El detalle que hace que esto funcione en un árbol de widgets es el valor de retorno booleano de `on_mouse_click`, `on_confirm` y `on_navigate`: no es “¿tuve éxito?” sino “¿consumí este evento?”. Un `Container` (la siguiente pieza) usa exactamente ese booleano para decidir si debe seguir intentando con otro hijo o detenerse, que es la misma pregunta que responde `gale.input_handler` a nivel de juego completo, pero ahora a nivel de cada widget individual. Un widget puramente decorativo (`Panel`, `Label`) no necesita sobrescribir ninguno de los cinco, pues su única responsabilidad es `render`. Su atributo de clase `focusable = False` le dice a cualquier contenedor que lo ignore al decidir a quién darle el foco de teclado.

9.7.2. Container: composición y enrutamiento

Container es, a la vez, un **Widget** más (tiene posición, tamaño y puede anidarse dentro de *otro Container*) y una lista ordenada de hijos. Su `on_mouse_click` recorre a sus hijos en orden *inverso* de inserción: el que se agregó último se prueba primero, porque es el que se dibuja encima. Se detiene en el primero que devuelva `True`:

```

1 def on_mouse_click(self, position, data) -> bool:
2     if not self.enabled or not self.contains(position):
3         return False
4
5     for child in reversed(self.children):
6         if not child.visible or not child.enabled:
7             continue
8         if child.on_mouse_click(position, data):
9             self._focus_only(child)
10            return True
11
12    return False

```

Ese `return True` temprano es exactamente lo que evita que un click sobre un botón “atraviese” hacia lo que hay detrás de él, pues en cuanto un hijo dice “esto era mío”, ningún otro hijo (ni el propio contenedor) lo vuelve a procesar. La navegación por teclado sigue una regla distinta, apropiada para listas y formularios en vez de superposición visual: solo se le pasa el evento al hijo que *ya* tiene el foco (`self._focused_child()`). Si ese hijo no lo consume (por ejemplo, un `ListView` que ya está en su última fila y no tiene a dónde moverse más), el contenedor interpreta el `on_navigate` no consumido como una petición de mover el foco al siguiente hermano enfocable. `gale.ui` recomienda explícitamente construir un menú como composición en vez de una clase nueva por pantalla:

```

1 menu = Container(40, 40, 240, 160, children=[
2     Panel(40, 40, 240, 160),
3     ListView(48, 48, 224, 144, items=[
4         ("Host", start_hosting), ("Join", start_joining),
5     ]),
6 ])

```

que es, de hecho, la misma receta que sigue `Menu` en `src/gui/Menu.py` de *Ultimate Fantasy*: un `Panel` (subclasificado solo para reproducir el bisel de dos capas del diseño original) más un `ListView` superpuesto. A eso se suma un `Cursor` adicional dibujado a mano en la posición de la fila seleccionada, porque el diseño del juego quiere el indicador de selección en una posición horizontal fija en vez del que `ListView` dibuja por defecto junto al texto.

9.7.3. Theme: apariencia centralizada

Cada widget expone una propiedad `theme` que, si no se le asignó un tema propio, cae de vuelta a `get_default_theme()`:

```

1 @property
2 def theme(self) -> Theme:
3     return self._theme if self._theme is not None else
        get_default_theme()

```

Un `Theme` es solo una bolsa de valores —fuente, colores de texto, fondo, borde, acento, *hover*, foco y deshabilitado, más un espaciado— con valores por defecto razonables. Como la propiedad relee `get_default_theme()` en cada acceso (nunca copia el tema en el momento de construir el widget), llamar a `set_default_theme` en cualquier momento cambia la apariencia de *todos* los widgets que no tengan un tema explícito, de inmediato, sin recrearlos. *Ultimate Fantasy* usa exactamente este mecanismo en `main.py`:

```

1 from gale.ui.theme import set_default_theme
2 from src.gui.theme import DEFAULT_THEME
3 ...
4 set_default_theme(DEFAULT_THEME)

```

fijando una sola vez, al arrancar, el borde blanco y el relleno gris oscuro que se ve en cada panel y barra de progreso del juego. Además reserva un segundo tema (`BAR_THEME`, con borde negro de 1px y sin relleno propio) solo para las barras de vida/experiencia, que a 3 píxeles de alto quedarían cubiertas casi por completo por el borde de 2 píxeles del tema por defecto. Esa es la ventaja concreta de centralizar el estilo en un objeto `Theme` en vez de repetir colores literales en cada `render`: cambiar la paleta del juego completo es cambiar un solo objeto, y un widget particular (como las barras) puede apartarse de esa paleta sin tocar el resto.

9.7.4. Button y ListView: dos lecturas del mismo contrato

`Button` ilustra el caso más simple de `Widget`: se activa tanto por click del mouse (`on_mouse_click`, solo al *soltar* el botón, no al presionarlo) como por `on_confirm` mientras tiene el foco. Ambos caminos terminan en el mismo método privado `_activate`, que invoca el *callback* `on_click` pasado al construirlo. `ListView`, la pieza que `Menu` reutiliza para construir cualquier menú del juego, generaliza esa misma idea a una columna de filas: `on_navigate` mueve `selected_index` con *wraparound* `((self.selected_index + dy) % len(self.items))`, `on_confirm` invoca el *callback* de la fila seleccionada, y `on_mouse_click/on_mouse_motion` traducen una posición de pantalla a un índice de fila con `_row_at`. Ninguno de los dos widgets sabe nada de sonido ni de las reglas particulares de *Ultimate Fantasy*. Es `Menu.navigate/Menu.confirm` quien decide reproducir el efecto "blip" *solo* cuando el `ListView` subyacente confirma haber consumido la navegación, envolviendo así el widget genérico con el detalle específico del juego sin modificar `gale.ui`:

```

1 def navigate(self, direction) -> None:
2     if self.list_view.on_navigate(direction):
3         settings.SOUNDS["blip"].stop()
4         settings.SOUNDS["blip"].play()

```

9.8. Persistencia de partidas: gale.save

El Capítulo 8 mencionó, al hablar del Family Computer Disk System, que guardar la partida fue la innovación técnica que hizo posible el diseño de mundo abierto de *The Legend of Zelda*. Pero ningún juego de este libro la había necesitado hasta ahora, ya que Princess se juega de una sentada, y todos los anteriores caben en una sola pantalla. Ultimate Fantasy es distinto –un mundo de cinco regiones, un elenco que sube de nivel, un combate largo– así que es el primer punto del libro donde “guardar y continuar después” deja de ser un lujo.

`SaveManager` resuelve esto sin que el juego tenga que diseñar su propio formato de archivo. Recibe cualquier diccionario serializable a JSON, lo guarda en un *slot* con nombre, y se encarga de la escritura atómica (un archivo temporal renombrado al final, para que una caída a mitad de escritura nunca deje un guardado corrupto) y del versionado del esquema. Lo único que le corresponde al juego es decidir *qué* guardar y cómo reconstruirlo.

9.8.1. Qué se guarda y qué se regenera

Region, en Ultimate Fantasy, se genera proceduralmente (Sección 7.2 del Capítulo 7) a partir de nada más que su configuración de puertas, que nunca cambia en tiempo de ejecución. Por eso el guardado no incluye el mapa de ninguna región, solo el nombre de en cuál está el jugador. Lo que sí es estado real, propio de esta partida y de ninguna otra, vive en `Party/Character`: nivel, experiencia, salud actual, y posición. Cada clase expone su propio par `to_dict()/load_dict()`, exactamente el mismo patrón en los tres niveles, `Character`, `Party`, `World`, cada uno delegando en el siguiente:

```

1 # World.to_dict
2 def to_dict(self) -> Dict[str, Any]:
3     return {
4         "current_region_name": self.current_region_name,
5         "party": self.party.to_dict(),
6     }
7
8 # Party.to_dict
9 def to_dict(self) -> Dict[str, Any]:
10    return {
11        "genders": self.party_genders,
12        "characters": {
13            str(k): character.to_dict()

```

```

14         for k, character in self.characters.items()
15     },
16 }

```

`Character.to_dict` guarda únicamente los campos mutables –nivel, experiencia, salud, estadísticas efectivas, posición– porque todo lo demás (nombre, textura, animaciones, clase) depende solo del género elegido para ese puesto del grupo, y `Party.genders` ya lo conserva. Reconstruir un personaje es, entonces, volver a construir el `Party` tal como al empezar una partida nueva y *después* sobrescribir sus campos mutables con `load_dict`, en vez de mantener un camino de reconstrucción paralelo y separado.

9.8.2. El menú de pausa: una pila dentro de la pila, otra vez

Guardar necesita un lugar en la interfaz desde donde dispararse, y ese lugar es, precisamente, el caso canónico de pila de estados que el Capítulo 8 dejó pendiente: un menú apilado *sobre* el mundo, que sigue visible y congelado debajo. `PauseMenuState` es un `Menu` de tres entradas, Continuar, Guardar partida, Salir, construido exactamente como `StatsMenuState` (Sección 8.3), y se apila sobre `PlayState` al presionar la tecla de pausa:

```

1 def on_input(self, input_id, input_data) -> None:
2     if input_id == "pause" and input_data.pressed:
3         self.state_machine.push(
4             PauseMenuState(self.state_machine), play_state=self
5         )
6         return
7
8     self.world.on_input(input_id, input_data)

```

“Guardar partida” llama de vuelta a `PlayState.save_game`, que es el único punto del juego que conoce a `SaveManager`:

```

1 def save_game(self) -> None:
2     SaveManager().save(settings.SAVE_SLOT, self.world.to_dict())

```

y, tras guardar, la propia `PauseMenuState` se saca a sí misma de la pila y apila en su lugar un `ShowTextState` –el mismo estado, sin ningún cambio, que ya anunciaba el nombre de cada región nueva en la Sección 7.2– con el texto “partida guardada”. Tres estados por un instante, uno encima del otro: el mundo, congelado, abajo del todo; nada en medio, porque el menú acaba de sacarse a sí mismo; y el aviso, que se desvanece solo.

9.8.3. Continuar desde la pantalla de título

Cargar exige la operación simétrica desde `StartState`: al entrar, pregunta si existe un guardado (`SaveManager().exists(...)`) y, si lo hay, habilita una segunda tecla junto a “Enter para nueva partida”:

```

1 def enter(self) -> None:
2     settings.play_music("intro")
3     self.has_save = SaveManager().exists(settings.SAVE_SLOT)
4
5 def _continue_game(self) -> None:
6     try:
7         save_data = SaveManager().load(settings.SAVE_SLOT)
8     except SaveError:
9         return
10
11     party_genders = {
12         int(k): v for k, v in save_data["party"]["genders"].items
13         ()
14     }
15     # ... funde a negro y apila PlayState con save_data=save_data

```

`PlayState.enter` acepta ahora un `save_data` opcional, y si está presente, construye el `World` exactamente como en una partida nueva y de inmediato le aplica `load_dict` encima, en vez de mantener dos caminos de construcción distintos para “partida nueva” y “partida cargada”.

A JSON no tiene claves enteras

`Party.to_dict` construye `{str(k): ...}`, con `str(k)` explícito, aunque `self.characters` use enteros (0, 1, ...) como clave. La razón es que el formato JSON solo permite cadenas de texto como clave de un objeto, así que cualquier serializador, el `json.dumps` que usa `gale.save` incluido, convierte silenciosamente toda clave entera a cadena al guardar. Si no se deshace ese cambio explícitamente al cargar, `self.characters[0]` deja de existir y en su lugar aparece una clave "0" nunca antes vista. Por eso `_continue_game` reconstruye `party_genders` con `int(k)` para cada clave, deshaciendo a mano la conversión que JSON hizo sin avisar.

9.9. Construcción de Ultimate Fantasy

Construcción completa, capa por capa: un mundo navegable con cinco regiones generadas proceduralmente, colisión de cerca y transición de región por los bordes; un sistema de entidades en dos niveles (`Entity/BattleEntity`) del que cuelgan `Character`, `Enemy` y `NPC`; y un grupo (`Party`) que mueve a sus personajes en formación y decide encuentros aleatorios sobre pasto alto. A eso se suma un combate por turnos completo, con menú de acción, selección de objetivo, resolución de daño/curación con componente aleatorio, experiencia y subida de nivel. La interfaz, por su parte, está construida enteramente con `gale.ui` (`Panel`, `ListView`, `ProgressBar`, `Cursor`) sobre un tema centralizado, y la persistencia de partidas usa `gale.save`, mediante un menú de pausa que se guarda y se continúa desde la pantalla

de título. El resultado es un flujo de pantallas —pueblo, mundo, combate, menús de acción/objetivo/estadísticas, pausa, pantalla de fin de juego— coordinado enteramente por una única `StateStack` raíz, sin que ningún estado individual necesite conocer la totalidad del árbol de pantallas del que forma parte.

9.10. Ejercicios propuestos

1. Añadir un nuevo tipo de acción de combate (por ejemplo, un objeto curativo usable desde un inventario durante el combate) que reutilice la máquina de estados de `TakeTurnState/SelectActionState` existente sin modificar su estructura general, pues basta con que la acción aparezca como una entrada adicional en la lista de `entity.actions` de quien la usa, con su propio `action["func"]`.
2. Diseñar, por escrito, la interfaz mínima (atributos y métodos) que debería cumplir cualquier entidad para participar en el sistema de combate, y verificar, leyendo `BattleEntity`, que `Character` y `Enemy` la cumplen sin duplicar ni un solo método entre ambas.
3. Implementar un nuevo widget de `gale.ui` propio (por ejemplo, un `Slider` numérico) que respete el contrato de `Widget` (`on_mouse_click/on_confirm/ on_navigate` devolviendo si consumió el evento) y pueda insertarse sin cambios dentro de un `Container` ya existente.
4. Extender `Region` para que, además de césped/cerca/pasto alto, admita una cuarta capa de “agua” intransitable, y decidir en qué capa debería vivir esa nueva regla de colisión sin duplicar la lógica que ya consulta la capa `"fence"` en `PartyWalkState`.

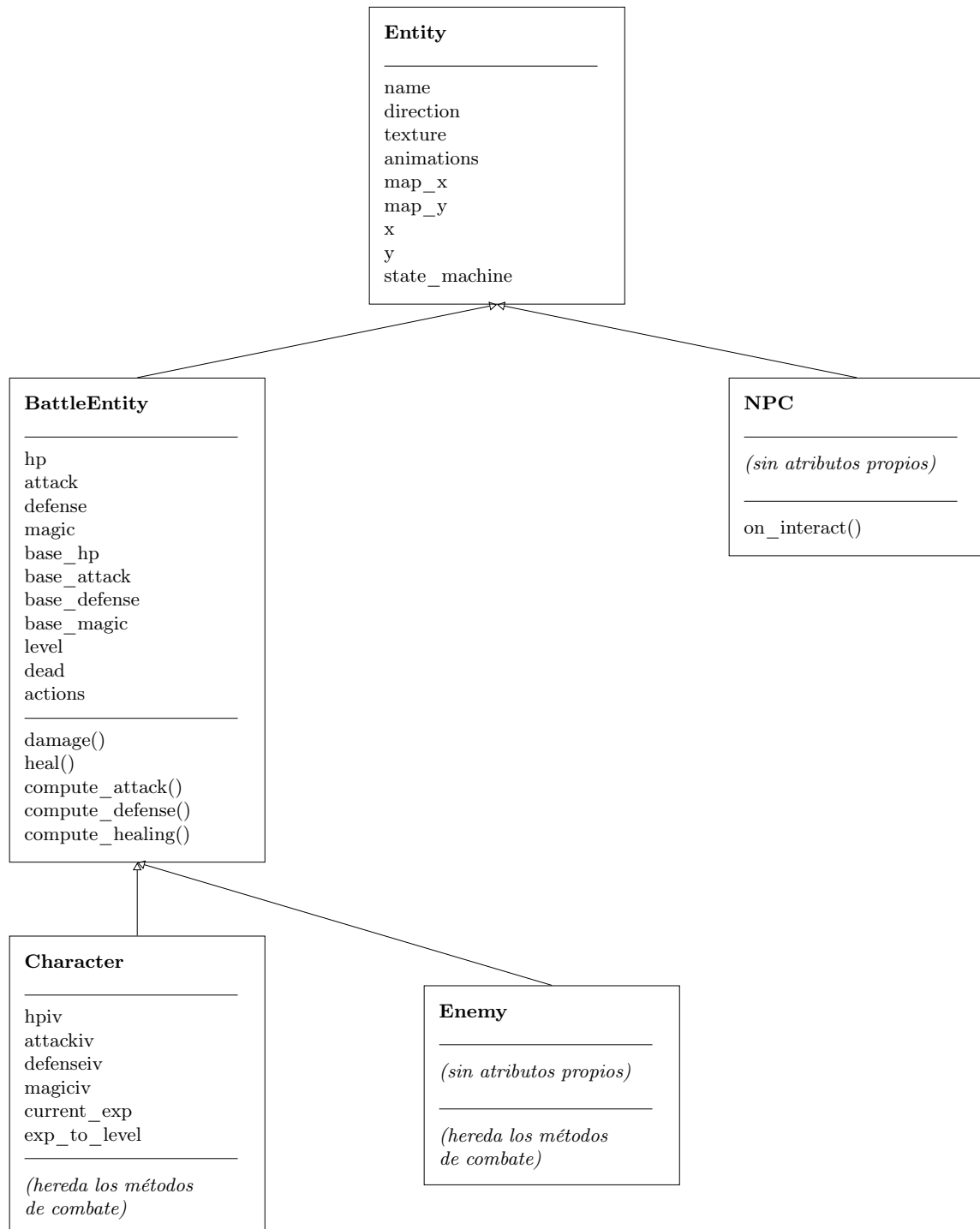


Figura 9.3: Jerarquía de entidades de Ultimate Fantasy: **BattleEntity** y **NPC** heredan de **Entity**, y **Character** y **Enemy** heredan a su vez de **BattleEntity**, la única rama que participa en combate.

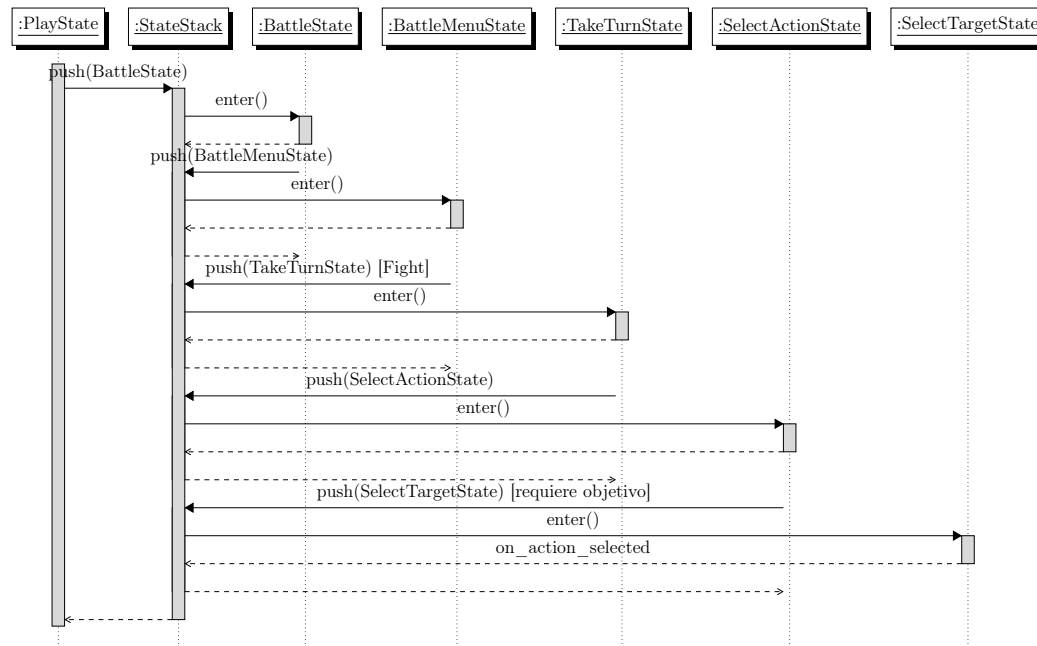


Figura 9.4: Diagrama de secuencia UML del combate contra el pasto alto: **PlayState** empuja **BattleState** sobre la **StateStack** sin abandonar su propia ejecución (su barra de activación permanece abierta durante todo el combate); elegir *Fight* en **BattleMenuState** empuja **TakeTurnState**, que a su vez empuja **SelectActionState** por cada combatiente, y una acción que exige objetivo empuja finalmente **SelectTargetState**. Cada barra de activación se abre con un **push** y solo se cierra cuando su estado hace **pop**, de modo que la profundidad de la pila, y no solo la del diagrama, crece con cada pantalla nueva.

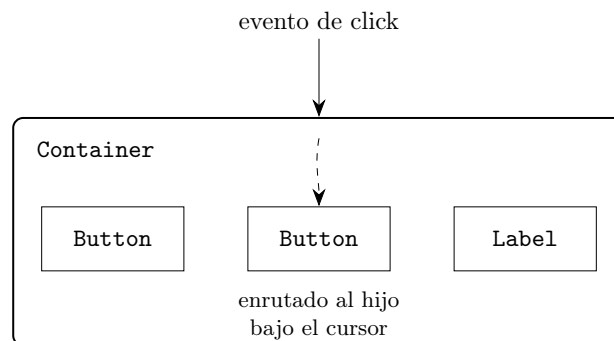


Figura 9.5: Enrutamiento de un click, que llega primero al **Container** y no a un widget en particular: **Container.on_mouse_click** recorre a sus hijos y enruta el evento (flecha punteada) únicamente al que está bajo el cursor, aquí el segundo **Button**, que lo consume sin que el **Label** ni el otro **Button** lleguen siquiera a evaluarlo.



Figura 9.6: Una partida en curso de *Ultimate Fantasy*.